

AI-Enhanced Guided-Inquiry Platform for Collaborative Learning in Introductory Programming

Bo Kim¹, James Skon²

¹ Department of Computer Science
Southern New Hampshire University
Manchester, NH 03106
b.kim@snhu.edu

² Kenyon College
Gambier, OH 43022
skonjp@kenyon.edu

Abstract

This paper presents a digital learning framework, coLearn-AI (hereafter referred to as the system), that combines guided inquiry pedagogy with intelligent, adaptive feedback mechanisms to support collaborative learning in introductory programming courses. The system enables instructors to author structured inquiry activities, manage student groups, and provide embedded AI guidance that scaffolds learning in real time. Students work in groups and engage with progressively complex problems through exploration, collaboration, and reflection. The system integrates synchronous collaborative workspaces, turn-taking mechanisms, and analytics dashboards that help instructors monitor student progress at scale. A key innovation is the integration of AI-driven formative feedback (OpenAI API is used), where instructors embed AI guidance directives within activity files. These directives, visible only to instructors, enable the AI module to provide context-aware feedback that aligns with pedagogical goals. By bridging structured human facilitation with adaptive, intelligent tutoring capabilities, the system promotes engagement, accountability, and deeper conceptual understanding. The platform has been successfully deployed in multiple university courses for in-class work since September 2025; instructors monitor progress through dashboards. We report implementation status and early classroom use. This paper describes the pedagogical foundations, system architecture, activity model, AI feedback pipeline, and group-learning mechanisms and outlines directions for formal efficiency evaluation and refinement.

1 Introduction

Introductory programming courses often face challenges, including uneven engagement, limited opportunities for individualized guidance, and difficulty sustaining collaborative learning at scale [1]. These challenges are amplified by the wide diversity of students' prior experiences, technical backgrounds, and cultural or social contexts. For many learners, particularly those with little or no programming experience, courses can feel intimidating when peers appear substantially more prepared, discouraging participation and help-seeking and exacerbating feelings of exclusion or impostor syndrome. Traditional lecture-based formats often fail to provide sufficient feedback or opportunities for students to articulate reasoning or collaborate with peers. Prior work in computing education has consistently shown that structured, collaborative, inquiry-based learning can reduce these barriers while improving conceptual understanding, retention, and problem-solving performance. One goal of our approach is to create a supportive environment where students can work together, leverage diverse perspectives, and engage with course material in a way that feels enjoyable rather than threatening.

At the same time, the emergence of generative artificial intelligence presents both opportunities and challenges. AI tools have the potential to provide immediate feedback, help students identify misconceptions, and scaffold their reasoning. However, unmanaged AI support may bypass learning objectives, provide over-assistance, or conflict with pedagogical intent. As a result, there is a need for learning environments where AI is tightly aligned with instructor goals, supports guided reasoning, and integrates naturally into collaborative activity structures.

To address these challenges, we developed an AI-enhanced guided inquiry platform that blends the pedagogical strengths of Process-Oriented Guided Inquiry Learning (POGIL) with a robust technical architecture for collaborative programming instruction. The system enables instructors to author guided-inquiry activities using a flexible markup language or Google Docs, embed hidden AI directives that shape the nature of automated feedback, and deploy these activities to student groups working synchronously in class.

Within the system, students work in small rotating groups, adopting structured roles (e.g., facilitator, spokesperson, quality controller, process analyst) that promote equitable participation, metacognitive reflection, and collaborative sensemaking. The system enforces turn-taking, tracks active participation, and synchronizes workspaces in real-time, allowing groups to collaborate through exploration, concept invention, application, and challenge cycles.

To ground the discussion that follows, **Figure 1** presents the system architecture and data flow, mapping instructor-facing services, storage, and AI mediation, as well as the student collaboration workspace with its feedback loop.

A distinguishing feature of the proposed system is its instructor-aligned AI feedback layer. Rather than providing unconstrained assistance, AI responses are shaped by hidden instructor-authored directives describing the depth, tone, and structure of feedback. This preserves the integrity of guided inquiry by encouraging students to explain their reasoning and evaluate alternatives without shortcutting the intellectual work of conceptual construction.

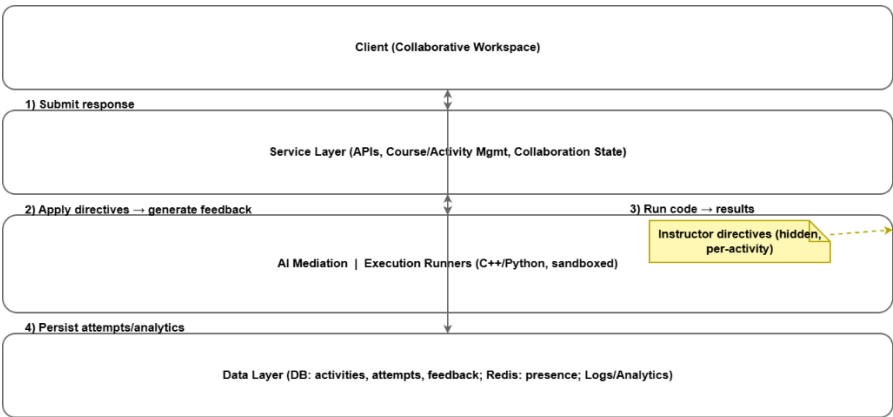


Figure 1. System Architecture and Data Flow. *A layered diagram depicting client, service layer, AI mediation, execution runners, and data stores; arrows show request/response paths for code execution and AI feedback.*

The system incorporates a secure execution environment for C++ and Python programming tasks, supports file I/O simulation, and logs student attempts for analytics purposes. Its backend architecture supports multi-course management, activity deployment, group monitoring, and data storage. Together, these pedagogical and technical components enable instructors to scale collaborative, inquiry-based instruction across many programming courses.

To preview how guided inquiry integrates with AI-mediated feedback in the proposed system, **Figure 2** illustrates the activity cycle, highlighting the AI touchpoints where responses are evaluated, nudges are generated, and analytics are logged.

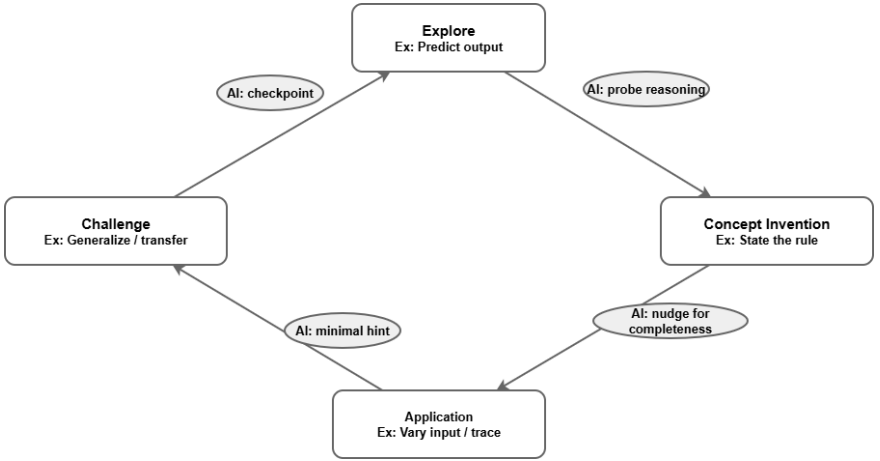


Figure 2. Guided Inquiry Cycle and AI Touchpoints. *The four inquiry phases (Explore → Concept Invention → Application → Challenge) are overlaid with AI interventions: directive-aware evaluation of submissions, generation of formative prompts, and logging/analytics for reflection and instructor monitoring.*

The remainder of this paper presents a comprehensive description of the proposed system, including its pedagogical foundations, system architecture, activity model, collaborative learning mechanisms, AI feedback pipeline, and ongoing research and development plans.

2 Related Work and Pedagogical Foundations

Research in computer science education consistently emphasizes active learning and structured student engagement as key factors in achieving improved outcomes. Guided-inquiry approaches, particularly those inspired by POGIL, have demonstrated effectiveness in fostering collaboration, conceptual understanding, and student agency in introductory courses [1]-[3]. These methods emphasize structured team roles, shared responsibility, and cycles of exploration, concept invention, application, and challenge.

Collaborative learning has been shown to enhance students' reasoning processes by promoting explanation, peer instruction, and negotiation of meaning; these effects are especially relevant for programming tasks that require articulation of strategy and justification of choices [4]. Inductive pedagogies (including inquiry) provide a general framework for sequencing experiences, allowing students to infer principles from evidence rather than receive them through exposition [5].

In programming education specifically, inquiry structures have been used to organize multi-stage reasoning and sustain engagement across problem sequences [6]. At the same time, work on AI in education and intelligent tutoring systems (ITS) underscores both the promise and the constraints of automation: systems can provide hints, formative feedback, and assessment at scale, but must be aligned to pedagogical goals to avoid counterproductive over-assistance [7], [8].

These strands motivate our design, which integrates guided inquiry, role-aware collaboration, and instructor-aligned AI within a single system.

2.1 Active, Inductive, and Collaborative Learning

Active learning and cooperative structures are associated with improved outcomes and equitable participation [1], [4]. Inductive pedagogies (inquiry-, problem-, and discovery-oriented) sequence experiences so learners infer principles from evidence rather than receive them solely by exposition [5]. For programming, this translates into tasks that require learners to externalize strategy, predict program behavior, test hypotheses, and justify revisions. We adopt this stance for code-centric tasks where articulating strategy, justifying predictions, and reconciling conflicting interpretations are essential.

2.2 Guided Inquiry / POGIL Lineage and Adaptation

POGIL supplies a well-studied template for structuring small-group work via explicit roles and phased activities [2], [3]. Our adaptation follows a four-phase sequence (Explore → Concept Invention → Application → Challenge) implemented in a digital environment with lightweight role rotation and visible turn histories. Teams work in groups of three or four; rotation is enforced gently to foster equitable participation while maintaining a natural feel in synchronous settings. Reflection prompts require students to cite program behavior as evidence, reinforcing metacognitive habits noted in prior work [3], [4]. (See **Figure 2**.)

2.3 Instructor-Aligned AI Support

The AI in the system is not intended to function as an autonomous tutor, but as a pedagogically constrained assistant. Its behavior is governed by hidden instructor-authored directives that specify the intended tone, level of scaffolding, and criteria for evaluating student work; prompts occur at defined touchpoints in the inquiry cycle (see **Figure 2**). In each step, students first attempt a response; the AI then issues targeted nudges, requesting a missing condition, probing for evidence, or re-anchoring to the prompt, after which teams revise. The goal is not simply to reach a “correct” answer, but to support students in collectively constructing well-reasoned explanations and solutions that reflect deepening understanding.

This approach ensures that

- feedback remains aligned with activity objectives,
- solutions are not revealed prematurely,
- misconceptions are surfaced and addressed productively, and
- guidance strengthens reasoning rather than encouraging answer-seeking.

This aligns with the findings of Lee and Moore [7], who emphasize the importance of instructor oversight and scaffolding in AI-mediated formative assessment.

2.4 Integrating Programming Work with Inquiry

Because many introductory concepts are best grasped by observing live behavior, the system incorporates secure execution for C++ and Python. Students explore, run, and revise code within the same collaborative activity where they justify predictions and reconcile differences in reasoning. The coupling of executable artifacts with conceptual prompts helps students connect rules and results, supporting debugging as a form of sense-making rather than trial and error [6]. Reflection prompts close the loop by asking learners to name the idea they now trust and how they would recognize or apply it in future work.

2.5 Gaps Addressed

Despite substantial progress in intelligent tutoring and auto-grading, most systems still optimize either individualized feedback or scalable assessment workflows, not the combination of structured collaboration, guided inquiry, and instructor-aligned AI. Systems that offer code execution often treat collaboration as peripheral (shared editors without pedagogical roles) or treat pedagogy as peripheral (auto-grading without inquiry). Likewise, AI feedback is often generic, providing hints that may be technically correct but pedagogically misaligned. Our system here addresses these gaps by binding three elements that are rarely integrated and creates an interactive, web-based learning environment: (i) a role-aware collaboration model that makes equitable participation a first-class goal, (ii) a guided-inquiry activity structure that sequences exploration, generalization, and transfer, and (iii) an AI feedback layer that is explicitly authored and constrained by the instructor [1], [3], [5], [7], [8].

Importantly, the proposed system is not intended to operate as an independent or self-contained tutor. Its effectiveness depends on knowledgeable instructors who design high-quality guided-inquiry activities and understand how to shape the learning process. The system amplifies, not replaces, expert pedagogy: AI guidance functions as an extension of the instructor's intentions, enabling scalable implementation of established learning practices rather than attempting to automatically teach arbitrary content. In this way, the system should be understood as an instructor-centered tool

that leverages AI to increase the efficacy and reach of well-designed learning experiences.

3 System Architecture

The system architecture translates the foregoing pedagogy into a responsive, multi-service platform capable of supporting many sections and courses simultaneously. As previewed in **Figure 1**, the client renders activities and synchronizes group work in real-time, while the server orchestrates activity delivery, AI mediation, execution jobs, and data storage. A modern web front-end coordinates collaborative editing, presence, and turn-taking; a service layer exposes APIs for course management, activity lifecycle, and analytics; and a relational database persists activities, rosters, attempts, and feedback. A collaboration service maintains low-latency shared state for group interactions. When a prompt involves code, the request is routed to containerized runners (C++/Python) that compile or execute programs with resource/time limits, capture outputs, and return structured results linked to the group's current step.

The AI mediation service sits alongside these components. When students submit a response, the server packages the response, the local context (e.g., earlier attempts, code output), and the instructor's embedded directives. The AI service interprets those directives to shape its response, deciding whether to probe, nudge, or request clarification, and whether to ask a follow-up or accept the response, allowing the group to move forward. All exchanges, such as student drafts, code runs, AI prompts, and final submissions, are logged for later instructor review and research analysis. Instructor dashboards also surface basic learning analytics indicators to support timely intervention and early warning [10].

3.1 Activity Model and AI Feedback Layer

Activities are the unit of design that binds pedagogy to infrastructure. Each activity is parsed into a structured representation of sections, question groups, prompts, expected answer features, optional rubrics, code blocks, and instructor notes. This representation enables the client to render rich interactions while giving the server sufficient structure to route code runs, validate formats, and coordinate the AI. (See **Figure 1** for the end-to-end pipeline; **Figure 2** shows where AI engages within the inquiry cycle.)

The AI feedback layer is invoked whenever a step carries an embedded directive. Rather than answering the question, the AI examines the student's response against the author's intentions. If a response appears partial, the AI might request a missing condition; if it seems correct but brittle, it might propose a counterexample; if it is off topic, it might re-anchor the group to the prompt. In a code step, the AI may point

out an observed discrepancy between the expected and actual output and request an explanation, rather than suggesting a wholesale fix. These behaviors are bounded by instructor-authored directives, enumerated in **Table 1**, so that feedback should remain formative, concise, and aligned with the author’s intent. To avoid stalls, instructors can enable a controlled progress-with-feedback path (e.g., “Checker errors should not block”), allowing teams to advance when only minor issues remain while logging signals for later review. This preserves momentum without compromising the focus of the inquiry.

When appropriate, the AI can also honor rubric language by flagging evidence of specific criteria and recording that evidence for the instructor’s analytics [7], [8].

Table 1. Examples of AI Directives and Pedagogical Intent *Directives are hidden in activity files and shape feedback scope, tone, and gating during each step of the inquiry cycle.*

Directive (hidden)	Operational Effect on Feedback	Pedagogical Intent
Follow-ups: gibberish-only	Provide guidance only when responses are empty, off-target, or incoherent; allow productive struggle otherwise	Minimize interruptions; respect productive struggle
Requirements-only	Highlight missing conditions; avoid giving strategies	Preserve inquiry; focus on completeness
Match sample output exactly	Enforce exact labeling/ordering on final checkpoint	Support assessment reliability
Checker errors do not block	Provide hints but allow progression after minor issues	Maintain momentum; avoid over-scaffolding
Concise, single-sentence feedback	Compress responses for fast rhythm in live collaboration	Reduce cognitive load; keep teams talking

Activities authored in a lightweight markup or imported from Google Docs are normalized into sections, prompts, expected features, rubrics, code blocks, and instructor notes, allowing the system to route execution, apply rubric logic, and coordinate AI (see **Table 1**).

3.2 AI Mediation: Model and Inference Specification

The platform uses a large language model (ChatGPT) as a formative-feedback assistant. The base model is trained and maintained by the provider on licensed, public, human-generated sources. We do not fine-tune; instead, we steer behavior through prompt conditioning grounded in instructor-authored directives. Course artifacts are stored in our database and supplied only at inference time; they are not used to train the base model.

At runtime, each feedback turn is built from a directive-aware prompt. A concise system role defines the assistant's function ("a teaching aide constrained by instructor directives and pedagogy"). Hidden directives authored by the instructor (such as Follow-ups: gibberish-only, Requirements-only, Concise feedback, Fail-open) specify tone, scaffolding bounds, and follow-up policy. The context window contains only the minimum needed to reason about the current step – activity and item identifiers, the prompt text, brief rubric cues, paraphrases of tests or exemplars (not answer keys), and small, quoted excerpts of the student's code or output. For open-practice steps, we use low-temperature sampling (temperature = 0.2) to produce succinct, targeted nudges. For stricter checkpoints, an even lower-temperature can be used. The service expects a structured reply: a one-to-three-sentence rationale that cites concrete evidence (e.g., line numbers or tokens) and, where configured, a compact set of feature flags indicating missing conditions or misconceptions. All responses are schema-validated and truncated server-side before storage or display.

Inference calls are issued by the Activity service against the OpenAI chat completions endpoint with per-request credentials, rate limiting, and latency/cost logging. The web client (single-page application) renders feedback inline during practice and suppresses hints during locked checkpoints while still collecting student justifications. All prompts and responses are recorded with step metadata to support reproducibility and instructor oversight.

Privacy safeguards are enforced server-side. Prompts exclude personally identifiable information (PII), including names, email addresses, IDs, and identifiable paths or comments. Only minimal excerpts needed for the current turn are transmitted. Vendor training on our prompts and responses is disabled where supported, and audit logs retain redacted prompts, model outputs, and user actions with timestamps for later review.

This mediation layer makes the AI component a documented, reproducible part of the system rather than a black box: model and interface are specified, inputs are minimized and redacted, outputs are constrained by instructor intent and validated for form and brevity. In combination with the activity model, it yields feedback that is predictable in tone, bounded in scope, and inspectable after the fact, and

amendable to the evaluation metrics reported later, while preserving the pedagogical primacy of guided inquiry.

3.3 Collaborative Group Model

The collaborative layer is designed to make equitable participation the default rather than an aspiration. **Figure 3** illustrates the collaborative group dynamics, including rotating roles, turn-taking, and participation signals used during each activity step. Each group’s workspace surfaces the “active student” and a transparent history of turns, so that orchestration is visible and contestable: students see when it is time to drive, and when to review. This visibility matters because the benefits of collaboration only appear when conversational obligations are distributed and when reasoning is externalized.

From the instructor’s perspective, a session is a moving field of small teams. The dashboard provides a real-time overview of all groups in class, showing which students are logged in, who is currently the active participant within each group, and how far each team has progressed through the assigned activity. Instructors can immediately identify stalled or struggling groups based on progress indicators, common errors, or repeated AI guidance triggers, and determine which teams may require additional support or intervention.

Because AI feedback is constrained by embedded directives, its prompts are predictable in tone and scope, allowing instructors to rely on it as a partner rather than an uncontrollable variable. Over multiple offerings of a course, instructors can compare runs of the same activity to see how different cohorts engaged, which items were consistently sticky, and how revisions to wording or exemplars affected learning dynamics [1], [6].

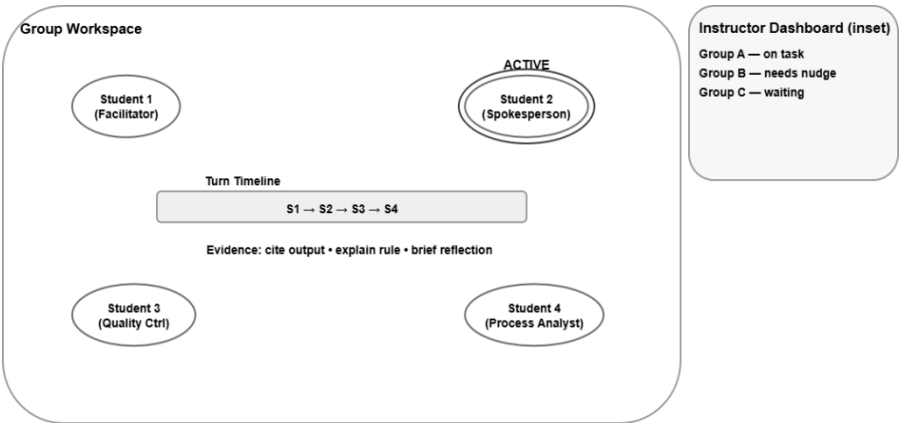


Figure 3. Group Collaboration Model. *An annotated sketch showing rotating “active” role, shared workspace, visible turn history, and instructor overview panel.*

Instructors can adopt the system incrementally, beginning with inquiry activities and reflections, then adding live execution, and later enabling rubric checkpoints or summative assessments. Reliability features (checkpointing, execution logs, and autoscaling runners) and basic analytics support timely interventions and early detection of at-risk patterns [10].

4 Evaluation Plan and Early Observations

A mixed-methods evaluation is planned, combining learning outcome measures with process analytics and qualitative accounts of classroom experience. The quantitative strand examines performance on near- and far-transfer items, changes in error profiles on representative code tasks, and retention across the term. The qualitative strand analyzes group transcripts, reflection responses, and instructor field notes to understand how guidance, role rotation, and AI prompts shape talk, explanation, and consensus-building [3], [4]. Preliminary observations suggest that visible turn-taking smooths participation and reduces dominance by a single strong student, while directive-aligned AI prompts are effective at eliciting missing conditions and reframing off-track reasoning without revealing complete solutions. These impressions align with prior findings that inquiry structures and group roles can enhance equity and conceptual engagement in programming contexts [6] and that AI feedback is most effective when its behavior is guided by pedagogy rather than left generic [7].

The system has been implemented and is in active use in multiple university course sections. The following summarizes the current status of the system implementation:

- Activity authoring (markup + Google Docs import) – implemented
- Grouping/turn-taking, real-time sync – implemented
- C++/Python runners (containerized, resource caps) – implemented
- AI mediation (ChatGPT via OpenAI API) – implemented
- Assessment mode (single-learner lock, timer, hint suppression) – implemented
- Instructor review with audit trail, publish/finalize – implemented
- Analytics warehouse & exports – partially implemented
- Optional LMS sync – planned
- Versioned item bank & regrade workflow – planned

We will also track how revisions to AI directives (e.g., shifting from “requirements-only” to “default” follow-ups, see **Table 1**) affect student behavior and solution quality across iterations. Dashboards will be augmented with early-warning indicators to support timely intervention [10].

It is essential to note that the proposed system does not eliminate the need for active instructor involvement. As with any group-oriented, collaborative activity, instructors should monitor engagement, observe how well students enact their assigned roles, and intervene when groups struggle with collaboration. Effective use of the system requires instructors to circulate, review group dashboards, and encourage equitable participation and serious engagement with each question.

5 Limitations and Future Work

Several limitations remain. First, alignment of AI feedback with instructor goals depends on the quality of directives; poorly specified guidelines can yield prompts that are too terse, too leading, or insufficiently sensitive to the activity’s aims. Second, even with role rotation, groups can drift into asymmetric participation; additional nudging mechanisms and analytics may be needed to maintain balanced contributions [4]. Third, while the current runners support C++ and Python well, broadening language coverage and improving multi-file project ergonomics will be important for upper-division use. Finally, we have only preliminary classroom experience; results to date are observational rather than causal. These limitations motivate a more rigorous evaluation and continued refinement of directive templates, authoring linting, and operational safeguards (e.g., autoscaling runners, offline fallbacks). Future work will therefore focus on (i) directive authoring tools with previews and “safety checks,” (ii) lightweight collaborative debugging views that make reasoning about state a shared activity, (iii) adaptive group-formation and rotation policies informed by process data, and (iv) evaluation study that assesses the efficiency of this pedagogical approach, such as learning achieved per unit of student and instructor time, comparing the system against strong baselines. The study will investigate whether directive-aligned guided inquiry yields faster mastery with equal or reduced instructional effort, while monitoring collaboration equity and hint dependence as process indicators [1], [4], [6].

6 Conclusion

The proposed system combines guided inquiry, equitable collaboration, and instructor-aligned AI into a single platform, making high-quality, discussion-rich programming instruction feasible at scale. AI, under explicit directives, becomes a reliable contributor to the learning process, capable of asking the questions an expert tutor would ask. By treating AI as a controlled pedagogical tool and by making participation structures visible and lightweight, the system supports reasoning, reflection, and transfer rather than simply providing answers. Early deployments suggest that the approach reduces instructional bottlenecks and deepens conceptual engagement, while the architecture provides a foundation for credible evaluation and iterative improvement. As institutions seek sustainable ways to blend human

facilitation with intelligent tools, the system offers a model where pedagogy and automation reinforce each other rather than compete.

References

- [1] S. Freeman, S. L. Eddy, M. McDonough, M. K. Smith, N. Okoroafor, H. Jordt, and M. P. Wenderoth, "Active learning increases student performance in science, engineering, and mathematics," *Proceedings of the National Academy of Sciences*, vol. 111, no. 23, pp. 8410–8415, 2014.
- [2] J. J. Farrell, R. S. Moog, and J. N. Spencer, "A guided-inquiry general chemistry course," *Journal of Chemical Education*, vol. 76, no. 4, pp. 570–574, 1999.
- [3] R. S. Moog and J. N. Spencer, *Process-Oriented Guided Inquiry Learning (POGIL)*. Oxford, U.K.: Oxford Univ. Press, 2008.
- [4] D. W. Johnson, R. T. Johnson, and K. A. Smith, "Cooperative learning: Improving university instruction by basing practice on validated theory," *Journal on Excellence in College Teaching*, vol. 25, no. 4, pp. 85–118, 2014.
- [5] M. J. Prince and R. M. Felder, "Inductive teaching and learning methods: Definitions, comparisons, and research bases," *Journal of Engineering Education*, vol. 95, no. 2, pp. 123–138, 2006.
- [6] S. Ahmed, S. A. H. Bukhari, and A. Ahmad, "Inquiry-based learning in software engineering education: Exploring students' multiple inquiry levels in a programming course," *Frontiers in Education*, vol. 10, Article 1503996, 2025.
- [7] S. S. Lee and R. L. Moore, "Harnessing generative AI (GenAI) for automated feedback in higher education: A systematic review," *Online Learning*, vol. 28, no. 3, pp. 82–104, 2024.
- [8] I. Roll and R. Wylie, "Evolution and revolution in artificial intelligence in education," *International Journal of Artificial Intelligence in Education*, vol. 26, no. 2, pp. 582–599, 2016.
- [9] W. Holmes, M. Bialik, and C. Fadel, *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Boston, MA: Center for Curriculum Redesign, 2019.
- [10] O. H. T. Lu, A. Y. Q. Huang, J. C. H. Huang, A. J. Q. Lin, H. Ogata, and S. J. H. Yang, "Applying learning analytics for the early prediction of students' academic performance in blended learning," *Educational Technology & Society*, vol. 21, no. 2, pp. 220–232, 2018.